

Projeto: Norte

Descrição

Sistema web de controle financeiro pessoal que permite aos usuários registrar, organizar e visualizar suas transações financeiras (entradas e saídas), com dashboard simples e intuitivo.

Objetivo do Projeto

Projeto pessoal para aprendizado e portfólio, focado em:

- aprender Micronaut e aplicar boas práticas de engenharia de software
 - aplicar na prática o máximo de matérias vistas na faculdade: requisitos de software, arquitetura de software, projeto de software, programação modular, teste de software, gerência de configuração, gerência de projeto, redes de computadores, sistemas operacionais, IHC (interação humano-computador) — e, como fase futura opcional, dev mobile
-

Stack Tecnológica

Backend

- Java 17+/21, Micronaut, Gradle
- Migrations versionadas com Flyway
- Documentação de API com OpenAPI/Swagger ([micronaut-openapi](#))

Banco de Dados

- PostgreSQL

Frontend

- React + TypeScript + Vite
- Tailwind CSS
- shadcn/ui (componentes sobre Radix UI)
- Framer Motion (animações e microinterações)
- Recharts (gráficos do dashboard)
- React Hook Form + Zod (formulários e validação)
- TanStack Query (estado de dados vindos da API)
- Cliente de API TypeScript gerado a partir do OpenAPI ([openapi-typescript](#) ou [orval](#))

Testes

- Backend: testes unitários (service) e de integração (repository/API), preferencialmente com Testcontainers
- Frontend: Vitest + React Testing Library (unitário), Playwright (e2e)

Infraestrutura / Deploy (custo zero)

- VM no **Oracle Cloud Free Tier** ("Always Free" — 2 OCPUs/12GB, permanentemente grátis, sem risco de cobrança)
- Domínio gratuito via **DuckDNS** (ex: norte.duckdns.org), evitando comprar domínio
- **Nginx** como proxy reverso na VM
- HTTPS gratuito com **Let's Encrypt** (Certbot)
- Docker / Docker Compose
- CI/CD com GitHub Actions (build + testes a cada push)

Design / UI

- Paleta: roxo, azul e preto — temática cyberpunk/futurista e tecnológica
- Fundo em "quase preto" (não preto puro), para reduzir cansaço visual
- Roxo como cor primária de interface (botões, navegação, estados ativos)
- Azul/ciano como destaque e também para sinalizar entradas (income) — reforça a sensação de confiança/calma esperada de um app financeiro
- Magenta/rosa vibrante para saídas (expense), mantendo tudo dentro do tema cyberpunk em vez de cair no verde/vermelho genérico
- Atenção a contraste e legibilidade dos valores financeiros (ligado à matéria de IHC)

Ferramentas

- DBeaver (gerenciamento do banco)
-

Requisitos Funcionais

- RF01: Cadastro, autenticação, edição e remoção de usuário
- RF02: CRUD de transações
- RF03: Dashboard com saldo, entradas e saídas
- RF04: Tipos de transação (INCOME / EXPENSE)
- RF05: Filtro de transações
- RF06: Gerar comprovante de transação (gerado no backend, formato PDF/imagem)
- RF07: Compartilhar comprovante (ex: WhatsApp, via Web Share API do navegador, com fallback de download)

Requisitos Não Funcionais

- RNF01: Autenticação com JWT

- RNF02: Senhas com hash
 - RNF03: Rotas protegidas
 - RNF04: Idempotência em operações críticas
 - RNF05: Validação de dados
 - RNF06: Performance
 - RNF07: Responsividade
 - RNF08: Acessibilidade (contraste, navegação por teclado, leitura por leitor de tela)
-

Modelo de Dados (Banco)

Tabela: users

- id (PK)
- name
- email (UNIQUE)
- password_hash
- phone (opcional)
- cpf (UNIQUE, opcional)

Tabela: categories

- id (PK)
- name
- user_id (FK → users.id)

Tabela: transactions

- id (PK)
- title
- amount (DECIMAL — nunca FLOAT/DOUBLE, para evitar erro de arredondamento)
- type (INCOME | EXPENSE)
- date
- user_id (FK → users.id)
- category_id (FK → categories.id, nullable, ON DELETE SET NULL)

Não é necessária tabela própria para o comprovante (RF06) — ele é gerado sob demanda a partir dos dados já existentes em **transactions**.

Relacionamentos

- User (1) → (N) Transactions
- User (1) → (N) Categories
- Category (1) → (N) Transactions

Regras importantes

- NÃO usar `income` e `expense` como colunas separadas — usar `amount` + `type`
- NÃO armazenar saldo no banco — saldo é calculado
- Evitar usar CPF como PK (usar `id`)

Convenções

- Tabelas no plural: `users`, `transactions`, `categories`
 - Classes no singular: `User`, `Transaction`, `Category`
-

Matérias da faculdade aplicadas

- **Requisitos de software** → RF/RNF formalizados, com critérios de aceite
 - **Arquitetura de software** → camadas documentadas (controller/service/repository), decisões registradas (ADRs)
 - **Projeto de software** → diagrama de classes, diagramas de sequência (login, criação de transação, geração/compartilhamento de comprovante)
 - **Programação modular** → separação em camadas, princípios SOLID
 - **Teste de software** → testes unitários e de integração no backend, unitários e e2e no frontend
 - **Gerência de configuração** → migrations versionadas, ambientes separados, Docker, variáveis de ambiente, pipeline de CI/CD
 - **Gerência de projeto** → board no GitHub Projects, roadmap dividido em fases
 - **Redes de computadores** → design de API REST/HTTP, deploy com proxy reverso e HTTPS
 - **Sistemas operacionais** → deploy em VM Linux, configuração de serviço, containers
 - **IHC (interação humano-computador)** → avaliação heurística, persona(s), teste de usabilidade com usuários reais, acessibilidade
 - **Dev mobile** (fase futura/stretch) → app com React Native reaproveitando a API
-

Roadmap por fases

Fase 0 — Preparação de ambiente

- Criar o repositório novo no GitHub com o nome **norte**
- Gerar o esqueleto do backend com o Micronaut CLI (`mn create-app`)
- Gerar o esqueleto do frontend com Vite (`npm create vite@latest`, template React + TypeScript)
- Subir um `docker-compose.yml` só com o PostgreSQL, para desenvolvimento local
- Criar um board no GitHub Projects com as fases como colunas/milestones

Fase 1 — Modelagem e diagramas (antes de codar)

Diagramas a produzir (salvar em `/docs/diagrams` no repositório):

1. **DER** — Diagrama Entidade-Relacionamento, seguindo o Modelo de Dados acima
2. **Diagrama de casos de uso** — RF01 a RF07 mapeados para os atores (usuário autenticado / não autenticado)
3. **Diagrama de arquitetura** — React → Controller → Service → Repository → PostgreSQL, mostrando o filtro JWT
4. **Diagrama de classes** — `User`, `Transaction`, `Category`, DTOs, `ReceiptService`
5. **Diagramas de sequência** — login com JWT; criação de transação com verificação de idempotência; geração e compartilhamento de comprovante
6. **Persona(s) de usuário**
7. **Fluxo de telas / wireframes** — Login, Cadastro, Dashboard, Lista de transações com filtro, Nova transação, Comprovante (já usando a paleta roxo/azul/preto)

Fase 2 — Backend base

- Configurar Flyway para migrations versionadas do banco
- Criar entidades (`User`, `Transaction`, `Category`) e DTOs
- Repository (Micronaut Data), Service (regras de negócio, cálculo de saldo), Controller (endpoints REST)
- Autenticação: cadastro, login, hash de senha (BCrypt), JWT, rotas protegidas
- Validação de dados (Bean Validation)
- Endpoint de geração de comprovante (`GET /transactions/{id}/receipt`)
- Documentação com OpenAPI/Swagger

Fase 3 — Testes de backend

- Testes unitários dos Services (saldo, idempotência)
- Testes de integração dos Repositories/Controllers (Testcontainers com Postgres)
- Cobertura dos fluxos principais (RF01 a RF07)

Fase 4 — Infraestrutura e CI/CD

- `Dockerfile` do backend e `docker-compose.yml` completo
- Pipeline no GitHub Actions: testes a cada push/PR, build da imagem
- Deploy: VM no Oracle Cloud Free Tier, domínio via DuckDNS, Docker instalado, Nginx como proxy reverso, HTTPS com Let's Encrypt

Fase 5 — Frontend

- Scaffold com Vite + React + TypeScript
- Tailwind CSS + shadcn/ui, tokens de design com a paleta roxo/azul/preto
- Cliente TypeScript gerado a partir do OpenAPI

- Telas: Login/Cadastro, Dashboard (saldo + gráficos), Lista de transações com filtro, Nova transação, Comprovante com botão de compartilhar (Web Share API + fallback de download)
- Estado de dados com TanStack Query
- Microinterações com Framer Motion
- Responsividade

Fase 6 — Testes de frontend

- Vitest + React Testing Library nos componentes principais
- Playwright cobrindo o fluxo e2e completo (login → criar transação → dashboard → gerar e compartilhar comprovante)

Fase 7 — IHC e acessibilidade

- Avaliação heurística (10 heurísticas de Nielsen), documentada
- Teste de usabilidade com 2-3 pessoas reais (script de tarefas, anotações, ajustes feitos)
- Checklist de acessibilidade (contraste via WebAIM, navegação por teclado, aria-labels)

Fase 8 — Documentação final

- README completo (o quê, por quê, como rodar, screenshots/gif, link do deploy)
- ADRs das decisões principais
- Diagrama de arquitetura atualizado, se algo mudou durante a implementação

Fase 9 — Stretch: app mobile

- App em React Native reaproveitando a mesma API e o mesmo cliente tipado
- Fluxo mínimo: login, dashboard, criar transação
- Só começar depois das fases 0–8 estarem sólidas

Ordem recomendada

0 → 1 (diagramas) → 2 (backend) → 3 (testes back) → 4 (infra/deploy) → 5 (frontend) → 6 (testes front) → 7 (IHC) → 8 (documentação) → 9 (mobile, opcional)